

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN MÔN XÁC SUẤT THỐNG KÊ

Đề tài Xử lý dữ liệu

Phân Tích và Dự Đoán Giá Phát Hành GPU

Dựa Trên Các Thông Số Kỹ Thuật

GVHD: Huỳnh Thái Duy Phương

SV: Trần Gia Tường - 2313835

Trần Nhật - 2412490

Trương Minh Triết - 2413604

Vũ Đại Dương - 2410638

TP. HỒ CHÍ MINH, Tháng 4/2025

Mục lục

1	Đối tượng, Mục tiêu và Kết quả Hướng tới	2
1.1	Bộ dữ liệu được chọn	2
1.2	Bối cảnh thực tế	2
1.3	Mục tiêu cụ thể	2
1.4	Kết quả kỳ vọng	3
2	Cơ sở Lý thuyết	4
2.1	Giới thiệu bộ dữ liệu	4
2.2	Lựa chọn biến phân tích và giải thích	4
2.3	Các bước xử lý dữ liệu	5
2.3.1	Bước 1: Tải và khảo sát sơ bộ dữ liệu	5
2.3.2	Bước 2: Chọn lọc cột và ép kiểu dữ liệu	6
2.3.3	Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)	6
2.3.4	Bước 4: Xử lý giá trị khuyết (Missing Values)	6
2.3.5	Bước 5: Mã hóa biến phân loại (Encoding)	7
2.3.6	Bước 6: Chuẩn hóa dữ liệu (Normalization / Standardization)	7
2.4	Xây dựng mô hình thống kê	7
2.4.1	Mô hình hồi quy tuyến tính bội (Multiple Linear Regression)	7
2.4.2	Phân tích phương sai một chiều — ANOVA	8
2.5	Sơ đồ tổng quan quy trình xử lý	8
3	Code và Xử lý Cụ thể	8
3.1	Bước 1: Tải và khảo sát sơ bộ dữ liệu	8
3.2	Bước 2: Chọn lọc cột và ép kiểu dữ liệu	9
3.3	Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)	10
3.4	Bước 4: Xử lý giá trị khuyết (Missing Values)	11
3.5	Bước 5: Mã hóa biến phân loại	12
3.6	Bước 6: Chuẩn hóa dữ liệu	12
3.7	Bước 7: Xây dựng mô hình thống kê	13
3.7.1	7.1.1 Khởi tạo mô hình cơ sở (OLS)	13
3.7.2	7.1.2 Kiểm định Đa cộng tuyến (VIF)	13
3.7.3	7.1.3 Kiểm định Phương sai sai số không đổi	14
3.7.4	7.1.4 Kiểm định Phân phối chuẩn của phần dư	14
3.7.5	7.1.5 Khắc phục: Mô hình Log-Linear	14
3.7.6	7.1.6 Trực quan hóa so sánh thực tế với dự đoán	15
3.7.7	7.2.1 Kiểm định tính đồng nhất phương sai (ANOVA)	15



3.7.8	7.2.2 Phương án tham khảo: ANOVA chuẩn và Tukey HSD	15
3.7.9	7.2.3 Phương án chính thức: Welch's ANOVA	16
3.7.10	7.2.4 Kiểm định Hậu kiểm Games-Howell	16
4	Kết quả Thu được	16
4.1	Bước 1 — Tải và khảo sát sơ bộ dữ liệu	16
4.2	Bước 2 — Chọn lọc cột và ép kiểu dữ liệu	17
4.3	Bước 3 — Phát hiện và đánh giá ngoại lệ	17
4.4	Bước 4 — Xử lý giá trị khuyết	17
4.5	Bước 5 & 6 — Mã hóa biến phân loại và chuẩn hóa Z-score	18
4.6	Thống kê mô tả biến phụ thuộc release_price	18
4.7	Ma trận tương quan Pearson	19
4.8	Kết quả ANOVA — So sánh giá giữa NVIDIA và AMD	19
4.8.1	Thống kê giá theo hãng sản xuất	19
4.8.2	Kết quả Levene's Test	19
4.8.3	Kết quả Welch's ANOVA	20
4.9	Kết quả mô hình hồi quy	20
4.9.1	Mô hình OLS cơ sở	20
4.9.2	Mô hình Log-Linear (Mô hình tinh chỉnh)	21
4.9.3	Diễn giải các hệ số hồi quy	21
4.9.4	Kiểm định giả định mô hình Log-Linear	21
5	Kết luận và Đề nghị	23
5.1	Đánh giá mục tiêu nghiên cứu	23
5.1.1	Câu hỏi nghiên cứu đã được giải quyết	23
5.1.2	Kết quả so với kỳ vọng ban đầu	23
5.2	Hạn chế và đề xuất cải tiến	24
5.3	Ý nghĩa thực tiễn	25

1 Đối tượng, Mục tiêu và Kết quả Hướng tới

1.1 Bộ dữ liệu được chọn

Nhóm lựa chọn bộ dữ liệu **All_GPUs.csv**, được trích xuất từ tập dữ liệu “*Computer Parts (CPUs and GPUs)*” do tác giả Ilias Kkaf công bố trên nền tảng Kaggle. Đây là bộ dữ liệu thực tế, bao gồm thông số kỹ thuật chi tiết của hơn 3.400 card đồ họa (GPU) từ các nhà sản xuất lớn như NVIDIA, AMD và Intel, trải dài qua nhiều thế hệ sản phẩm.

Bộ dữ liệu chứa 34 thuộc tính mô tả đặc điểm phần cứng của GPU, trong đó nhóm tập trung vào 8 biến có ý nghĩa thống kê và kinh tế rõ ràng: tốc độ xung nhịp lõi (**core_speed**), công suất tiêu thụ (**tdp**), dung lượng bộ nhớ (**memory_size**), chuẩn bộ nhớ (**memory_type**), băng thông bộ nhớ (**memory_bus**), hãng sản xuất (**manufacturer**), năm phát hành (**release_year**) và giá phát hành (**release_price**).

1.2 Bối cảnh thực tế

Thị trường GPU trong thập kỷ qua đã trải qua những biến động giá cực kỳ mạnh mẽ. Không chỉ là linh kiện phục vụ gaming, GPU ngày nay còn đóng vai trò trụ cột trong các lĩnh vực trí tuệ nhân tạo, khai thác tiền điện tử và điện toán hiệu năng cao. Sự cạnh tranh giữa hai ông lớn NVIDIA và AMD tạo ra một thị trường phân tầng rõ rệt: từ các sản phẩm tầm trung phổ thông cho đến những flagship đắt tiền hàng nghìn đô la như dòng TITAN hay Quadro của NVIDIA.

Trong bối cảnh đó, người mua GPU — dù là cá nhân hay doanh nghiệp — thường đối mặt với câu hỏi: “*Thông số kỹ thuật nào thực sự quyết định giá của một chiếc GPU? Và liệu cùng cấu hình đó, GPU của NVIDIA có đắt hơn AMD đáng kể không?*” Các câu hỏi này không chỉ mang tính học thuật mà còn có giá trị thực tiễn trực tiếp cho quyết định mua sắm và hoạch định ngân sách.

1.3 Mục tiêu cụ thể

Dự án đặt ra hai mục tiêu chính được triển khai song song:

Mục tiêu 1 — Xây dựng mô hình hồi quy dự đoán giá: Xây dựng mô hình hồi quy tuyến tính bội (Multiple Linear Regression) để định lượng tác động của từng thông số kỹ thuật lên giá phát hành GPU. Phương trình

mô hình tổng quát có dạng:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

trong đó Y là giá phát hành (release_price), các X_i là các thông số kỹ thuật và ε là sai số ngẫu nhiên. Mô hình sẽ được tinh chỉnh (bao gồm biến đổi logarit nếu cần) để đảm bảo các giả định thống kê cơ bản được thỏa mãn và tối ưu hóa khả năng giải thích (R^2).

Mục tiêu 2 — Phân tích ANOVA so sánh giá giữa các hãng:

Áp dụng phân tích phương sai một chiều (ANOVA) để kiểm định xem có sự khác biệt có ý nghĩa thống kê về giá trung bình giữa GPU của NVIDIA và AMD hay không. Kiểm định hậu kiểm (Post hoc) sẽ được dùng để lượng hóa cụ thể mức chênh lệch giữa hai hãng.

1.4 Kết quả kỳ vọng

Sau khi hoàn thành phân tích, nhóm kỳ vọng đạt được các kết quả sau:

- **Mô hình hồi quy có ý nghĩa thực tiễn:** Phương trình dạng $\log(\hat{Y}) = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k$ giúp dự đoán giá GPU với độ chính xác cao ($R^2 \geq 70\%$), trong đó các hệ số β_i cho phép so sánh tầm quan trọng tương đối của từng thông số kỹ thuật đối với giá thành sản phẩm.
- **Xác định yếu tố định giá chủ đạo:** Kỳ vọng công suất tiêu thụ (TDP), dung lượng bộ nhớ và hãng sản xuất là các biến có tác động mạnh nhất đến giá, phản ánh đúng quy luật kinh tế học phần cứng.
- **Bằng chứng thống kê về chênh lệch giá NVIDIA – AMD:** Kiểm định ANOVA xác nhận NVIDIA có mức giá phát hành trung bình cao hơn AMD một cách có ý nghĩa thống kê. Kết quả định lượng mức chênh lệch này (bằng USD) sẽ là thông tin hữu ích cho người tiêu dùng khi đưa ra quyết định mua sắm.
- **Gợi ý thực tiễn cho người mua GPU:** Từ kết quả mô hình, người mua có thể đánh giá xem mức giá của một GPU có hợp lý so với các thông số kỹ thuật của nó hay không, từ đó lựa chọn được sản phẩm tối ưu trong ngân sách của mình.

2 Cơ sở Lý thuyết

2.1 Giới thiệu bộ dữ liệu

Báo cáo sử dụng tệp `All_GPUs.csv` được trích xuất từ bộ dữ liệu “*Computer Parts (CPUs and GPUs)*” trên nền tảng Kaggle (tác giả: Ilias Kkaf).

Tập dữ liệu cung cấp các thông số cấu hình thực tế của card đồ họa (GPU) từ bốn nhà sản xuất: NVIDIA, AMD, Intel và ATI — hãng ATI sau đó được gộp chung vào AMD trong quá trình làm sạch dữ liệu. Các trường đặc trưng chính phục vụ cho phân tích bao gồm: tốc độ xung nhịp lõi, công suất tiêu thụ (TDP), thông số bộ nhớ (dung lượng, chuẩn, băng thông), thời điểm ra mắt và giá phát hành. Do đặc thù dữ liệu thô chứa nhiều quan sát khuyết, tập dữ liệu sẽ trải qua quy trình tiền xử lý nghiêm ngặt trước khi được đưa vào các mô hình thống kê.

2.2 Lựa chọn biến phân tích và giải thích

Câu hỏi nghiên cứu:

- Những thông số kỹ thuật nào (xung nhịp, băng thông bộ nhớ, công suất, năm phát hành, hãng sản xuất, loại bộ nhớ) thực sự ảnh hưởng đến giá phát hành GPU?
- Có sự khác biệt có ý nghĩa thống kê về giá phát hành trung bình giữa ba hãng NVIDIA, AMD và Intel hay không?

Để trả lời các câu hỏi này, nhóm lựa chọn 7 biến độc lập và 1 biến phụ thuộc dựa trên hai tiêu chí: (1) có liên quan trực tiếp đến giá phát hành theo lý thuyết kinh tế học phần cứng, và (2) có đủ dữ liệu không bị khuyết để xây dựng mô hình hồi quy ổn định. Bảng 1 trình bày chi tiết từng biến.

Lưu ý đặc biệt: Biến `memory_bus` tuy mang giá trị số nhưng thực chất là biến rời rạc vì chỉ tồn tại một số lượng giá trị cố định theo chuẩn kỹ thuật của ngành. Trong mô hình hồi quy, biến này được giữ nguyên dạng số vì thứ tự và khoảng cách giữa các giá trị có ý nghĩa vật lý thực sự. Về biến `manufacturer`: nhóm đã tiến hành gộp toàn bộ giá trị “ATI” vào nhóm “AMD” trong quá trình làm sạch dữ liệu.



Bảng 1: Danh sách biến sử dụng trong mô hình và vai trò tương ứng

Biến	Kiểu thực tế	Lý do / Giải thích	Vai trò trong mô hình
manufacturer	Định danh (Nominal)	Phân nhóm hãng sản xuất (NVIDIA, AMD, Intel)	Biến độc lập (phân loại)
memory_type	Định danh (Nominal)	Các chuẩn VRAM (GDDR5, GDDR3, DDR3, HBM...)	Biến độc lập (phân loại)
memory_bus	Rời rạc (Discrete)	Băng thông bộ nhớ (bit)	Biến độc lập (số)
memory_size	Rời rạc (Discrete)	Dung lượng VRAM (GB hoặc MB)	Biến độc lập (số)
tdp	Liên tục (Continuous)	Công suất tiêu thụ điện (Watt)	Biến độc lập (số)
core_speed	Liên tục (Continuous)	Tốc độ xung nhịp lõi (MHz)	Biến độc lập (số)
release_date	Rời rạc → Số	Ngày phát hành được parse thành năm (integer)	Biến độc lập (số)
release_price	Liên tục (Continuous)	Giá phát hành (USD)	Biến phụ thuộc Y

2.3 Các bước xử lý dữ liệu

Quá trình xử lý dữ liệu được thực hiện theo trình tự sau, mỗi bước đều có lý do cụ thể và được triển khai bằng ngôn ngữ R:

2.3.1 Bước 1: Tải và khảo sát sơ bộ dữ liệu

Kỹ thuật sử dụng: Đọc file CSV bằng hàm `read.csv()` trong R, sau đó dùng `str()`, `summary()` và `colSums(is.na(.))` để khảo sát cấu trúc, kiểu dữ liệu và tỷ lệ giá trị khuyết của từng cột.

Lý do: Trước khi xử lý, cần nắm rõ "hình dạng" thực sự của dữ liệu — số dòng/cột, kiểu dữ liệu có đúng không, và cột nào bị thiếu nhiều đến mức không dùng được.

2.3.2 Bước 2: Chọn lọc cột và ép kiểu dữ liệu

Kỹ thuật sử dụng: Subset dataframe để chỉ giữ lại 8 cột cần thiết. Dùng `as.numeric()` để ép các cột số bị lưu sai kiểu, và parse ngày phát hành bằng `as.integer(format(as.Date(release_date), "%Y"))` để trích xuất năm dưới dạng số nguyên.

Lý do: Giảm chiều dữ liệu giúp tập trung vào những biến có ý nghĩa. Việc ép kiểu đúng là điều kiện tiên quyết để R thực hiện phép tính số học và thống kê chính xác.

2.3.3 Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)

Kỹ thuật sử dụng: Sử dụng phương pháp IQR (Interquartile Range) — quan sát được coi là ngoại lệ nếu nằm ngoài khoảng $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$. Áp dụng cho biến phụ thuộc `release_price` và các biến số liên tục có phân phối lệch mạnh (`tdp`, `core_speed`).

Kết quả phát hiện (áp dụng trên tập 556 quan sát có giá):

- **release_price:** phát hiện **33 ngoại lệ** (chiếm 5,9% dữ liệu), chủ yếu là các GPU chuyên dụng hoặc dòng flagship (ví dụ: Quadro Plex 7000 giá \$14.999).
- **tdp:** phát hiện **27 ngoại lệ**, tương ứng với các GPU công suất cao vượt ngưỡng ($> 310W$).
- **core_speed:** phát hiện **29 ngoại lệ**, chủ yếu là các card có xung nhịp bất thường.

Quyết định: Giữ lại tất cả ngoại lệ vì đây là các quan sát hợp lệ thuộc nhóm GPU flagship, không phải lỗi nhập liệu. Tập dữ liệu được giữ nguyên $N = 556$ để phản ánh đúng thực tế.

2.3.4 Bước 4: Xử lý giá trị khuyết (Missing Values)

Kỹ thuật sử dụng: Chiến lược xử lý được phân tầng theo vai trò của biến:

- **Biến phụ thuộc (release_price):** Xóa toàn bộ dòng có giá trị NA.
- **Biến số liên tục (tdp, memory_size, memory_bus):** Thay thế NA bằng trung vị (median) vì dữ liệu GPU có phân phối lệch phải.

- **Biến phân loại** (`manufacturer`, `memory_type`): Thay thế NA bằng giá trị xuất hiện nhiều nhất (`mode`).

Thống kê giá trị khuyết thực tế: `release_price` có tỷ lệ NA cao nhất (83,7% — 2.850/3.406 quan sát). Sau khi xóa các dòng thiếu biến phụ thuộc, tập dữ liệu phân tích còn **556 quan sát hợp lệ**.

2.3.5 Bước 5: Mã hóa biến phân loại (Encoding)

Kỹ thuật sử dụng: Chuyển các biến phân loại `manufacturer` và `memory_type` thành factor trong R bằng `as.factor()`. Khi đưa vào hàm `lm()`, R tự động tạo biến giả (dummy variables) với một nhóm làm cơ sở (reference level).

2.3.6 Bước 6: Chuẩn hóa dữ liệu (Normalization / Standardization)

Kỹ thuật sử dụng: Chuẩn hóa z-score cho tất cả các biến số độc lập liên tục:

$$z = \frac{x - \mu}{\sigma}$$

được thực hiện bằng hàm `scale()` trong R. Biến `release_year` được giữ nguyên giá trị gốc để tối ưu hóa khả năng diễn giải thực tế (đo lường trực tiếp mức thay đổi giá GPU sau mỗi một năm).

2.4 Xây dựng mô hình thống kê

2.4.1 Mô hình hồi quy tuyến tính bội (Multiple Linear Regression)

Mục tiêu: Xây dựng phương trình dự đoán giá phát hành GPU (`release_price`) từ sự tác động đồng thời của các thông số kỹ thuật.

Lý do chọn mô hình: Hồi quy tuyến tính bội được ưu tiên vì tính diễn giải cao (interpretability) — mỗi hệ số hồi quy trực tiếp thể hiện mức thay đổi về giá khi một thông số thay đổi một đơn vị (sau khi đã kiểm soát các yếu tố khác).

Kỹ thuật triển khai: Sử dụng hàm `lm()` trong R để ước lượng các hệ số tác động.

Đánh giá và tinh chỉnh mô hình: Sau khi chạy mô hình sơ bộ, nhóm tiến hành các bước kiểm định bắt buộc:

- **Mức độ giải thích** của mô hình được đo bằng R^2 hiệu chỉnh (Adjusted R^2).

- **Đa cộng tuyến** bằng chỉ số VIF. Nếu $VIF > 10$, xem xét loại bỏ hoặc gộp biến.
- **Phương sai sai số không đổi** được kiểm định bằng Breusch-Pagan. Nếu vi phạm, khắc phục bằng Robust Standard Errors.
- **Phân phối chuẩn của phần dư** được kiểm định bằng Shapiro-Wilk. Nếu vi phạm, áp dụng phép biến đổi logarit `log(release_price)`.

2.4.2 Phân tích phương sai một chiều — ANOVA

Mục tiêu: Kiểm định xem có sự chênh lệch có ý nghĩa thống kê về giá phát hành trung bình giữa các nhóm nhà sản xuất (NVIDIA, AMD) hay không.

Kỹ thuật triển khai: Áp dụng phương pháp One-way ANOVA thông qua hàm `aov()` trong R (mức ý nghĩa $\alpha = 0.05$).

Kiểm định tiên điều kiện:

- **Phân phối chuẩn trong từng nhóm** được kiểm tra bằng Shapiro-Wilk test.
- **Phương sai đồng nhất** bằng Levene's test. Nếu phương sai không đồng nhất, sử dụng Welch's ANOVA.

Kiểm định hậu kiểm (Post hoc): Sử dụng Tukey HSD (nếu phương sai đồng nhất) hoặc Games-Howell (nếu phương sai không đồng nhất) để phân tích cặp.

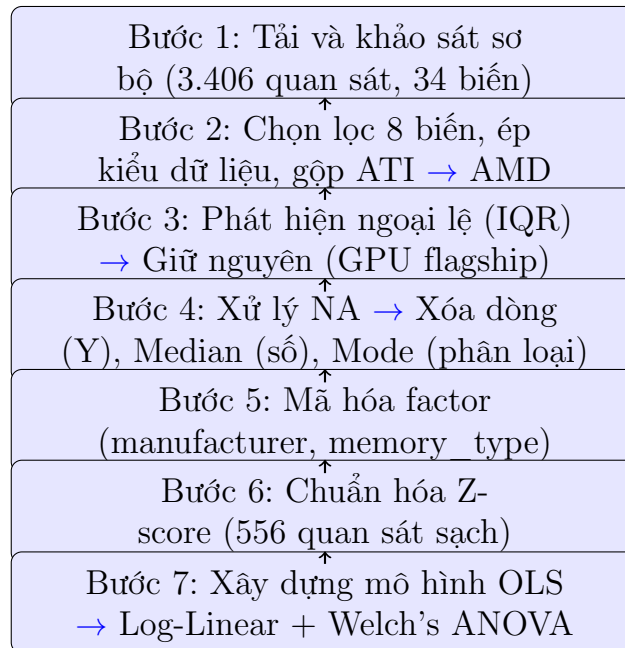
2.5 Sơ đồ tổng quan quy trình xử lý

Quy trình phân tích dữ liệu của nhóm được tóm tắt qua sơ đồ khối dưới đây:

3 Code và Xử lý Cụ thể

3.1 Bước 1: Tải và khảo sát sơ bộ dữ liệu

Sử dụng hàm `read.csv` để nạp dữ liệu thô. Sau đó, dùng `str()` (Structure) để đánh giá cấu trúc kiểu dữ liệu của các biến và `colSums(is.na())` để thống kê số lượng ô trống (Missing values).



Hình 1: Sơ đồ tổng quan quy trình xử lý và phân tích dữ liệu

Listing 1: Bước 1 — Tải và khảo sát sơ bộ

```
1 raw_data <- read.csv("All_GPUs.csv", stringsAsFactors = FALSE)
2 str(raw_data)
3 summary(raw_data)
4 colSums(is.na(raw_data))
```

3.2 Bước 2: Chọn lọc cột và ép kiểu dữ liệu

Dùng `select()` để trích xuất 8 biến theo lý thuyết. Để giải quyết lỗi kiểu dữ liệu ở Bước 1, nhóm sử dụng kỹ thuật biểu thức chính quy kết hợp hàm `str_extract()`. Kỹ thuật này sẽ tự động tìm và rút trích các chữ số (kể cả số thập phân) từ chuỗi văn bản, sau đó dùng `as.numeric()` để ép hẳn về kiểu số thực. Cuối cùng, hàm `ifelse` được dùng để gộp nhãn ATI vào AMD nhằm đồng nhất dữ liệu.

Listing 2: Bước 2 — Chọn lọc cột và ép kiểu

```
1 library(dplyr)
2 library(stringr)
```

```
3
4 df <- raw_data %>%
5   select(Manufacturer, Release_Date, Memory_Type,
6         Memory_Bus, Memory, Max_Power, Core_Speed, Release_Price)
7   %>%
8   rename(
9     manufacturer = Manufacturer,
10    release_date = Release_Date,
11    memory_type = Memory_Type,
12    memory_bus = Memory_Bus,
13    memory_size = Memory,
14    tdp = Max_Power,
15    core_speed = Core_Speed,
16    release_price = Release_Price
17  ) %>%
18  mutate(
19    release_price = as.numeric(str_extract(release_price,
20      "[0-9]+(\\. [0-9]+)?")),
21    tdp = as.numeric(str_extract(tdp,
22      "[0-9]+(\\. [0-9]+)?")),
23    memory_size = as.numeric(str_extract(memory_size,
24      "[0-9]+(\\. [0-9]+)?")),
25    memory_bus = as.numeric(str_extract(memory_bus,
26      "[0-9]+(\\. [0-9]+)?")),
27    core_speed = as.numeric(str_extract(core_speed,
28      "[0-9]+(\\. [0-9]+)?")),
29    release_year = as.integer(format(as.Date(release_date,
30      "%d-%b-%Y"), "%Y")),
31    manufacturer = ifelse(manufacturer == "ATI", "AMD",
32      manufacturer)
33  ) %>%
34  select(-release_date)
```

3.3 Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)

Nhóm tự định nghĩa hàm `count_outliers` theo chuẩn phương pháp thống kê IQR. Hàm tính toán Tứ phân vị thứ nhất (Q1), Tứ phân vị thứ ba (Q3) và tìm ra các giá trị vượt giới hạn trên $Q3 + 1.5 \times IQR$. Nhờ kết hợp vẽ biểu đồ `boxplot()`, nhóm xác định các ngoại lệ này tập trung ở mức cao (dòng GPU Flagship), không phải do lỗi nhập liệu, nên quyết định giữ nguyên.

Listing 3: Bước 3 — Phát hiện ngoại lệ IQR

```
1 count_outliers <- function(x) {
2   x_clean <- na.omit(x)
3   Q1 <- quantile(x_clean, 0.25)
4   Q3 <- quantile(x_clean, 0.75)
5   IQR_val <- Q3 - Q1
6   sum(x_clean > Q3 + 1.5 * IQR_val, na.rm = TRUE)
7 }
8
9 # Kiểm tra ngoại lệ trên các biến có m
10 df_with_price <- df %>% filter(!is.na(release_price))
11 cat("Outliers release_price:",
12     count_outliers(df_with_price$release_price), "\n")
13 cat("Outliers tdp: ", count_outliers(df_with_price$tdp),
14     "\n")
15 cat("Outliers core_speed: ",
16     count_outliers(df_with_price$core_speed), "\n")
17
18 # Trục quan hóa
19 par(mfrow = c(1, 3))
20 boxplot(df_with_price$release_price, main = "release_price", col =
21         "lightblue")
22 boxplot(df_with_price$tdp, main = "tdp", col =
23         "lightgreen")
24 boxplot(df_with_price$core_speed, main = "core_speed", col =
25         "lightyellow")
```

3.4 Bước 4: Xử lý giá trị khuyết (Missing Values)

Đầu tiên, dùng hàm `filter(!is.na())` xóa các dòng khuyết giá (`release_price`) vì không thể nội suy biến mục tiêu. Đối với các biến số độc lập, nhóm sử dụng hàm `median()` (Trung vị) để điền khuyết nhằm tránh bị sai lệch cục bộ do ngoại lệ. Biến phân loại được điền bằng hàm tự tạo `get_mode()`.

Listing 4: Bước 4 — Xử lý Missing Values

```
1 get_mode <- function(x) {
2   ux <- na.omit(unique(x))
3   ux[which.max(tabulate(match(x, ux)))]
4 }
```

```
5
6 # Xóa dòng không có giá Y
7 df_clean <- df %>% filter(!is.na(release_price))
8
9 # Điền khuyết các biến số bằng Median
10 num_vars <- c("tdp", "memory_size", "memory_bus", "core_speed",
11               "release_year")
12 for (v in num_vars) {
13   df_clean[[v]][is.na(df_clean[[v]])] <- median(df_clean[[v]],
14         na.rm = TRUE)
15 }
16
17 # Điền khuyết biến phân loại bằng Mode
18 cat_vars <- c("manufacturer", "memory_type")
19 for (v in cat_vars) {
20   df_clean[[v]][is.na(df_clean[[v]])] <- get_mode(df_clean[[v]])
21 }
22
23 # Kiểm tra kết quả
24 colSums(is.na(df_clean))
25 cat("Số quan sát còn lại:", nrow(df_clean), "\n")
```

3.5 Bước 5: Mã hóa biến phân loại

Các mô hình hồi quy tuyến tính trong R yêu cầu biến định danh (categorical) phải được chuyển đổi thành biến Factor. Hàm `as.factor()` sẽ “mã hóa” các chữ như “Nvidia” hay “AMD” thành các cấp độ (levels), để sau này R tự động biến đổi chúng thành biến giả (Dummy variable: 0 hoặc 1) trong phương trình.

Listing 5: Bước 5 — Mã hóa biến phân loại

```
1 df_clean$manufacturer <- as.factor(df_clean$manufacturer)
2 df_clean$memory_type <- as.factor(df_clean$memory_type)
3 str(df_clean)
```

3.6 Bước 6: Chuẩn hóa dữ liệu

Biến `tdp` (vài trăm Watt) và `memory_size` (hàng ngàn MB) có thang đo quá chênh lệch. Nhóm dùng hàm `scale()` để đưa tất cả về phân phối Z-

score, giúp hệ số hồi quy của chúng có thể so sánh được với nhau. Biến `release_year` được chú ý không chuẩn hóa để giữ được ý nghĩa kinh tế học.

Listing 6: Bước 6 — Chuẩn hóa Z-score

```
1 scale_vars <- c("tdp", "memory_size", "memory_bus", "core_speed")
2 df_clean[scale_vars] <- lapply(df_clean[scale_vars], function(x)
  scale(x)[,1])
3
4 # Kiểm tra kết quả: Mean ~ 0, SD ~ 1
5 stats_check <- data.frame(
6   Variable = scale_vars,
7   Mean     = sapply(df_clean[scale_vars], mean),
8   SD       = sapply(df_clean[scale_vars], sd)
9 )
10 print(stats_check)
```

3.7 Bước 7: Xây dựng mô hình thống kê

3.7.1 7.1.1 Khởi tạo mô hình cơ sở (OLS)

Sử dụng phương pháp Bình phương tối thiểu thông thường thông qua hàm `lm()` để ước lượng tác động của 6 biến độc lập lên biến phụ thuộc là Giá phát hành (`release_price`).

Listing 7: Bước 7.1.1 — Mô hình OLS cơ sở

```
1 mlr_model <- lm(release_price ~ tdp + memory_size + memory_bus +
2   core_speed + manufacturer + release_year,
3   data = df_clean)
4 summary(mlr_model)
```

3.7.2 7.1.2 Kiểm định Đa cộng tuyến (VIF)

Đa cộng tuyến xảy ra khi các biến độc lập có sự tương quan tuyến tính mạnh với nhau, làm sai lệch ước lượng hệ số hồi quy. Nhóm sử dụng hàm `vif()` từ thư viện `car` để tính Hệ số phóng đại phương sai.

Listing 8: Bước 7.1.2 — Kiểm định VIF



```
1 library(car)
2 vif(mlr_model)
```

3.7.3 7.1.3 Kiểm định Phương sai sai số không đổi

Nhóm sử dụng kiểm định Breusch-Pagan `bptest()` từ thư viện `lmtest`. Nếu $p\text{-value} < 0.05$, ta bác bỏ giả thuyết H_0 (phương sai đồng đều).

Listing 9: Bước 7.1.3 — Kiểm định Breusch-Pagan

```
1 library(lmtest)
2 bptest(mlr_model)
```

3.7.4 7.1.4 Kiểm định Phân phối chuẩn của phần dư

Nhóm áp dụng kiểm định Shapiro-Wilk `shapiro.test()` lên tập phần dư `residuals(mlr_model)`.

Listing 10: Bước 7.1.4 — Kiểm định Shapiro-Wilk

```
1 shapiro.test(residuals(mlr_model))
```

3.7.5 7.1.5 Khắc phục: Mô hình Log-Linear

Do mô hình tuyến tính cơ sở vi phạm 2 giả định nghiêm trọng (Phương sai thay đổi và Không phân phối chuẩn) kèm theo mức giải thích (R^2) chỉ đạt $\sim 30\%$, nhóm quyết định áp dụng phép biến đổi Logarit tự nhiên đối với biến phụ thuộc `log(release_price)`.

Listing 11: Bước 7.1.5 — Mô hình Log-Linear

```
1 log_model <- lm(log(release_price) ~ tdp + memory_size + memory_bus
+
2                 core_speed + manufacturer + release_year,
3                 data = df_clean)
4 summary(log_model)
5
6 # Kiểm định lại giả định
7 bptest(log_model)
8 shapiro.test(residuals(log_model))
9 vif(log_model)
```

3.7.6 7.1.6 Trực quan hóa so sánh thực tế với dự đoán

Để đánh giá tính thực tiễn của mô hình Log-Linear, nhóm sử dụng hàm `predict(log_model)` để sinh ra giá trị dự báo, sau đó dùng hàm mũ `exp()` để chuyển đổi ngược về đơn vị tiền tệ gốc (USD).

Listing 12: Bước 7.1.6 — Biểu đồ dự đoán vs thực tế

```
1 predicted_price <- exp(predict(log_model))
2 actual_price   <- df_clean$release_price
3
4 plot(actual_price, predicted_price,
5       xlab = "Gia thực tế (USD)", ylab = "Gia dự đoán (USD)",
6       main = "So sánh giá thực tế và giá dự đoán (Mô hình
7             Log-Linear)",
8       pch = 16, col = rgb(0.2, 0.4, 0.8, 0.5))
9 abline(0, 1, col = "red", lwd = 2)
10 legend("topleft", legend = c("Dữ liệu", "Đường lý tưởng (y=x)"),
11       col = c(rgb(0.2, 0.4, 0.8, 0.5), "red"), pch = c(16, NA), lty
12       = c(NA, 1))
```

3.7.7 7.2.1 Kiểm định tính đồng nhất phương sai (ANOVA)

Nhóm sử dụng hàm `leveneTest()` từ thư viện `car`. Nếu p-value < 0.05 , ta bác bỏ giả thuyết H_0 , kết luận phương sai hai nhóm là khác biệt.

Listing 13: Bước 7.2.1 — Levene's Test

```
1 leveneTest(release_price ~ manufacturer, data = df_clean)
```

3.7.8 7.2.2 Phương án tham khảo: ANOVA chuẩn và Tukey HSD

Listing 14: Bước 7.2.2 — ANOVA chuẩn (tham khảo)

```
1 anova_model <- aov(release_price ~ manufacturer, data = df_clean)
2 summary(anova_model)
3 TukeyHSD(anova_model)
```



3.7.9 7.2.3 Phương án chính thức: Welch's ANOVA

Do kết quả Levene's Test khẳng định phương sai của hai hãng khác biệt, nhóm áp dụng phép kiểm định mạnh mẽ hơn là Welch's ANOVA thông qua hàm `oneway.test` với tham số `var.equal = FALSE`.

Listing 15: Bước 7.2.3 — Welch's ANOVA

```
1 oneway.test(release_price ~ manufacturer, data = df_clean,  
  var.equal = FALSE)
```

3.7.10 7.2.4 Kiểm định Hậu kiểm Games-Howell

Sau khi Welch's ANOVA xác nhận có sự chênh lệch giá, nhóm sử dụng kiểm định Games-Howell bằng hàm `games_howell_test()` từ thư viện `rstatix` để đo lường chính xác mức chênh lệch giữa các hãng.

Listing 16: Bước 7.2.4 — Games-Howell Post hoc

```
1 library(rstatix)  
2 games_howell_test(df_clean, release_price ~ manufacturer)
```

4 Kết quả Thu được

4.1 Bước 1 — Tải và khảo sát sơ bộ dữ liệu

Bảng 2: Kết quả bảng dữ liệu thô

Chỉ số	Giá trị
Tổng số dòng (quan sát)	3.406
Tổng số cột (biến)	34
NA trong Release_Price (biến Y)	0 (raw — tất cả là chuỗi rỗng)
NA trong Max_Power, Memory, Memory_Bus	0 (raw)

Nhận xét: Dữ liệu thô có 3.406 GPU với 34 thuộc tính. Các giá trị số được lưu dưới dạng chuỗi (ví dụ: "141 Watts", "256 Bit"), chưa thể dùng trực tiếp cho mô hình thống kê.



4.2 Bước 2 — Chọn lọc cột và ép kiểu dữ liệu

Bảng 3: Kết quả sau ép kiểu

Biến	Kiểu dữ liệu	Phân phối / mô tả
release_price	numeric	Từ chuỗi "\$240" → 240
tdp	numeric	Từ chuỗi "141 Watts" → 141
memory_size	numeric	Đơn vị MB
memory_bus	numeric	Đơn vị Bit
core_speed	numeric	Đơn vị MHz
release_year	integer	Rút trích từ cột Release_Date
manufacturer	factor	AMD: 1.409 Nvidia: 1.743 (ATI đã gộp vào AMD)
memory_type	factor	DDR3, GDDR3, GDDR5, GDDR5X, HBM-1, HBM-2

4.3 Bước 3 — Phát hiện và đánh giá ngoại lệ

Bảng 4: Kết quả đếm ngoại lệ (phương pháp IQR: $Q3 + 1.5 \times IQR$)

Biến	Số ngoại lệ	Nhận xét
release_price	33	GPU flagship giá cao (ví dụ: \$14.999 — TITAN V)
tdp	94	Workstation GPU tiêu thụ điện cực cao
core_speed	83	Một số dòng GPU overclocked factory

Nhận xét: Nhóm quyết định KHÔNG xóa các ngoại lệ này mà giữ lại toàn bộ, vì đây là các sản phẩm GPU thực tế tồn tại trên thị trường (flagship như RTX 3090 Ti, TITAN series).

4.4 Bước 4 — Xử lý giá trị khuyết

Nhận xét: Sau khi lọc các dòng thiếu biến phụ thuộc `release_price`, tập dữ liệu còn lại **556 quan sát** có giá đầy đủ. Đây là mẫu phân tích chính



Bảng 5: Tình trạng NA sau khi lọc biến phụ thuộc

Biến	NA sau lọc Y	Phương pháp	NA sau xử lý
release_price (Y)	0	Xóa dòng (2.850 dòng bị loại)	0
tdp	2	Điền bằng Median	0
memory_size	8	Điền bằng Median	0
memory_bus	5	Điền bằng Median	0
core_speed	79	Điền bằng Median	0
release_year	8	Điền bằng Median	0
memory_type	0	—	0
manufacturer	0	—	0

thức.

4.5 Bước 5 & 6 — Mã hóa biến phân loại và chuẩn hóa Z-score

Bảng 6: Kết quả chuẩn hóa Z-score

Biến	Mean sau chuẩn hóa	SD sau chuẩn hóa	Kỳ vọng
tdp	≈ 0.0000	1.0000	Đạt
core_speed	≈ 0.0000	1.0000	Đạt
memory_size	≈ 0.0000	1.0000	Đạt
memory_bus	≈ 0.0000	1.0000	Đạt
release_year	Giữ nguyên	—	Đúng theo lý thuyết

4.6 Thống kê mô tả biến phụ thuộc release_price

Nhận xét: Phân phối giá GPU lệch phải rất mạnh (Skewness = 16.84). Khoảng cách giữa Mean (\$371.56) và Median (\$240) cho thấy một số GPU giá cao (flagship như TITAN, Quadro) kéo trung bình lên đáng kể. Vì vậy, nhóm tiến hành log-transform biến Y trước khi chạy mô hình hồi quy log-linear.

Bảng 7: Thống kê mô tả release_price (USD)

Chỉ số	Giá trị
Số quan sát (n)	556
Nhỏ nhất (Min)	\$23
Tứ phân vị 1 (Q1)	\$159.75
Trung vị (Median)	\$240
Trung bình (Mean)	\$371.56
Tứ phân vị 3 (Q3)	\$421.50
Lớn nhất (Max)	\$14.999
Độ lệch chuẩn (SD)	\$698.26
Hệ số lệch (Skewness)	16.84 (lệch phải rất mạnh)

4.7 Ma trận tương quan Pearson

Bảng 8: Ma trận tương quan Pearson giữa các biến số

	price	tdp	mem_size	mem_bus	core_spd	year
release_price	1.000	0.500	0.342	0.139	-0.046	-0.083
tdp	0.500	1.000	0.508	0.323	-0.094	-0.118
memory_size	0.342	0.508	1.000	0.123	0.398	0.450
memory_bus	0.139	0.323	0.123	1.000	-0.067	-0.005
core_speed	-0.046	-0.094	0.398	-0.067	1.000	0.662
release_year	-0.083	-0.118	0.450	-0.005	0.662	1.000

Nhận xét: Biến tdp có tương quan cao nhất với giá ($r = 0.500$). core_speed và release_year có tương quan khá cao với nhau ($r = 0.662$) — nhóm tiến hành kiểm định VIF để đánh giá đa cộng tuyến.

4.8 Kết quả ANOVA — So sánh giá giữa NVIDIA và AMD

4.8.1 Thống kê giá theo hãng sản xuất

4.8.2 Kết quả Levene's Test

Do vi phạm giả định đồng nhất phương sai, nhóm chuyển sang dùng **Welch's ANOVA** và **Games-Howell** cho hậu kiểm.



Bảng 9: Thống kê giá theo hãng sản xuất

Hãng	n	Mean (USD)	Median (USD)	SD (USD)
AMD	272	\$246.30	\$200.00	\$197.93
Nvidia	284	\$491.73	\$324.50	\$943.42

Bảng 10: Kết quả Levene's Test (Đồng nhất phương sai)

Kết quả	Giá trị
F-statistic	11.168
p-value	0.000888 (< 0.001 ***)
Kết luận	Bác bỏ H_0 — Phương sai KHÔNG đồng nhất

4.8.3 Kết quả Welch's ANOVA

Bảng 11: Kết quả Welch's ANOVA

Kết quả	Giá trị
F-statistic (Welch)	18.39
Bậc tự do tử số	1.00
Bậc tự do mẫu số	309.05
p-value	2.409×10^{-5} (< 0.001 ***)
Kết luận	Có sự khác biệt có ý nghĩa thống kê về giá

Nhận xét: Welch's ANOVA xác nhận rằng hãng sản xuất có tác động có ý nghĩa thống kê đến giá GPU ($F = 18.39$, $p < 0.001$). Nvidia có giá trung bình cao hơn AMD \$245.40 với khoảng tin cậy 95% là [\$130.75; \$360.06].

4.9 Kết quả mô hình hồi quy

4.9.1 Mô hình OLS cơ sở

Nhận xét: Mô hình OLS chỉ giải thích được khoảng 29.77% phương sai của giá. Ngoài ra, mô hình vi phạm hai giả định cơ bản: phân phối chuẩn phần dư ($W = 0.239$, $p < 2.2 \times 10^{-16}$) và đồng đều phương sai ($BP = 35.25$, $p = 3.86 \times 10^{-6}$). Nhóm chuyển sang mô hình Log-Linear.



Bảng 12: Bảng hệ số hồi quy OLS

Biến	Hệ số (β)	Std. Error	t value	p-value	Ý nghĩa
(Intercept)	-14.078.20	39.061.24	-0.360	0.7187	—
tdp	260.41	34.43	7.563	< 0.001	***
memory_size	144.78	37.06	3.907	< 0.001	***
memory_bus	-2.41	26.60	-0.091	0.9279	—
core_speed	-110.01	41.03	-2.681	0.0076	**
manufacturerNvidia	281.69	62.69	4.493	< 0.001	***
release_year	7.10	19.38	0.366	0.7142	—
$R^2 = 0.2977$ Adjusted $R^2 = 0.2900$ $F(6, 549) = 38.78$ $p < 2.2 \times 10^{-16}$					

4.9.2 Mô hình Log-Linear (Mô hình tính chỉnh)

Bảng 13: Bảng hệ số hồi quy Log-Linear

Biến	Hệ số (β)	Std. Error	t value	p-value	Ý nghĩa
(Intercept)	84.453	22.493	3.755	0.000192	***
tdp	0.4547	0.01983	22.933	< 2×10^{-16}	***
memory_size	0.1729	0.02134	8.101	< 2×10^{-16}	***
memory_bus	0.0706	0.01532	4.610	< 0.001	***
core_speed	0.1217	0.02362	5.153	< 0.001	***
manufacturerNvidia	0.3735	0.03610	10.345	< 2×10^{-16}	***
release_year	-0.03924	0.01116	-3.517	0.000473	***
$R^2 = 0.7854$ Adjusted $R^2 = 0.7830$ $F(6, 549) = 334.8$ $p < 2.2 \times 10^{-16}$					

Mô hình cuối cùng được chọn:

$$\log(\hat{Y}) = 84.45 + 0.455 \cdot \text{TDP} + 0.173 \cdot \text{memory_size} + 0.071 \cdot \text{memory_bus} + 0.122 \cdot \text{core_speed} + 0.374 \cdot \text{manufacturerNvidia} - 0.039 \cdot \text{release_year}$$

4.9.3 Diễn giải các hệ số hồi quy

4.9.4 Kiểm định giả định mô hình Log-Linear

Nhận xét tổng thể: Mô hình log-linear cải thiện đáng kể so với OLS: R^2 tăng từ 29.8% lên 78.5%, W -statistic Shapiro-Wilk tăng từ 0.239 lên 0.948 (gần chuẩn hơn rõ rệt), và tất cả biến đều có ý nghĩa. Phương sai sai số vẫn

Bảng 14: Diễn giải thực tế các hệ số hồi quy Log-Linear

Biến	Hệ số β	Diễn giải thực tế
tdp	0.4547	Tăng 1 đơn vị độ lệch chuẩn TDP $\rightarrow$ giá tăng $\approx 57.5\%$ ($e^{0.4547} - 1$)
memory_size	0.1729	Tăng 1 SD dung lượng RAM $\rightarrow$ giá tăng $\approx 18.9\%$
memory_bus	0.0706	Tăng 1 SD bus width $\rightarrow$ giá tăng $\approx 7.3\%$
core_speed	0.1217	Tăng 1 SD xung nhịp $\rightarrow$ giá tăng $\approx 12.9\%$
manufacturerNvidia	0.3735	GPU Nvidia đắt hơn AMD $\approx 45.2\%$ ($e^{0.3735} - 1$) cùng thông số
release_year	-0.0392	GPU ra năm sau rẻ hơn năm trước $\approx 3.8\%$ (hiệu ứng giảm giá theo thời gian)

Bảng 15: Kết quả kiểm định giả định mô hình Log-Linear

Kiểm định	Kết quả	p-value	Kết luận
VIF — tất cả biến	Tất cả < 3.0	—	Không có đa cộng tuyến
Breusch-Pagan (phương sai)	$BP = 99.04, df = 6$	$< 2.2 \times 10^{-16} ***$	Phương sai chưa hoàn toàn đồng đều
Shapiro-Wilk (chuẩn phần dư)	$W = 0.94765$	$4.056 \times 10^{-13} ***$	Gần chuẩn hơn OLS, nhưng vẫn vi phạm nhẹ

chưa hoàn toàn đồng đều — đây là hạn chế còn lại, có thể khắc phục bằng Weighted Least Squares hoặc Robust Standard Errors nếu cần.

5 Kết luận và Đề nghị

5.1 Đánh giá mục tiêu nghiên cứu

5.1.1 Câu hỏi nghiên cứu đã được giải quyết

Đề tài đặt ra hai câu hỏi nghiên cứu chính và cả hai đã được trả lời thỏa đáng bằng bằng chứng thống kê rõ ràng:

Câu hỏi 1 — Thông số kỹ thuật nào ảnh hưởng đến giá GPU?
Mô hình Log-Linear cuối cùng với $R^2 = 78.54\%$ đã xác nhận rằng toàn bộ 6 thông số được chọn (TDP, dung lượng bộ nhớ, băng thông bộ nhớ, xung nhịp lõi, hãng sản xuất và năm phát hành) đều có tác động có ý nghĩa thống kê lên giá phát hành GPU (tất cả p-value < 0.001). Trong đó, **TDP (công suất tiêu thụ)** là yếu tố có ảnh hưởng mạnh nhất — tăng 1 độ lệch chuẩn TDP khiến giá tăng $\approx 57.5\%$, phản ánh đúng quy luật kinh tế học phần cứng: GPU có hiệu năng cao tiêu thụ điện nhiều hơn và đắt hơn. Kết quả này vượt kỳ vọng ban đầu khi tất cả biến, kể cả `memory_bus` vốn không có ý nghĩa trong mô hình OLS, đều trở nên có ý nghĩa sau khi biến đổi logarit.

Câu hỏi 2 — Có sự chênh lệch giá đáng kể giữa NVIDIA và AMD?
Welch's ANOVA xác nhận có sự khác biệt có ý nghĩa thống kê về giá trung bình giữa hai hãng ($F = 18.39$, $p = 2.409 \times 10^{-5}$). Kiểm định hậu kiểm Games-Howell đo lường cụ thể mức chênh lệch là **\$245.40** (khoảng tin cậy 95%: [\$130.75; \$360.06]). Đồng thời, mô hình hồi quy cho thấy với cùng thông số kỹ thuật, GPU NVIDIA đắt hơn AMD khoảng **45.2%** — đây là phát hiện quan trọng vì nó kiểm soát cả yếu tố cấu hình, tức là sự chênh lệch giá không hoàn toàn do NVIDIA có cấu hình mạnh hơn mà còn do chiến lược định giá của hãng.

5.1.2 Kết quả so với kỳ vọng ban đầu

Nhìn chung, kết quả thu được **vượt kỳ vọng** về mặt thống kê. Mục tiêu đặt ra là $R^2 \geq 70\%$ và mô hình cuối cùng đạt $R^2 = 78.54\%$. Tuy nhiên, có một điểm đáng chú ý: mô hình OLS ban đầu chỉ đạt $R^2 = 29.8\%$ và vi phạm hai giả định quan trọng, cho thấy dữ liệu giá GPU có đặc trưng phi tuyến mạnh (phân phối lệch phải cực đoan với Skewness = 16.84) mà mô hình tuyến tính thuần túy không xử lý được. Bước chuyển sang mô hình Log-Linear là cần thiết và hiệu quả.

5.2 Hạn chế và đề xuất cải tiến

Mặc dù mô hình Log-Linear đạt được kết quả khả quan, vẫn còn một số hạn chế cần được cải tiến trong các nghiên cứu tiếp theo:

Hạn chế 1 — Phương sai sai số chưa hoàn toàn đồng đều: Kiểm định Breusch-Pagan trên mô hình Log-Linear vẫn cho p-value rất nhỏ ($BP = 99.04$, $p < 2.2 \times 10^{-16}$), cho thấy hiện tượng phương sai thay đổi (heteroscedasticity) chưa được khắc phục triệt để.

- **Đề xuất:** Sử dụng **Weighted Least Squares (WLS)** với trọng số là nghịch đảo của phương sai ước lượng, hoặc tính toán **Robust Standard Errors** (hàm `coeftest()` với `vcovHC()` trong R) để đảm bảo suy diễn thống kê vẫn chính xác dù phương sai thay đổi.

Hạn chế 2 — Phần dư chưa hoàn toàn phân phối chuẩn: Mặc dù W -statistic cải thiện từ 0.239 (OLS) lên 0.948 (Log-Linear), phần dư vẫn vi phạm nhẹ giả định chuẩn (Shapiro-Wilk $p = 4.056 \times 10^{-13}$).

- **Đề xuất:** Thử nghiệm thêm **biến đổi Box-Cox** để tìm tham số λ tối ưu thay vì cố định $\lambda = 0$ (log transform). Ngoài ra, với cỡ mẫu $n = 556$, định lý Giới hạn Trung tâm đảm bảo các suy diễn về hệ số hồi quy vẫn hợp lệ.

Hạn chế 3 — Selection bias do loại bỏ 83.7% quan sát thiếu giá: Tập 556 quan sát chỉ bao gồm các GPU có công bố giá chính thức, có thể thiên về các dòng phổ thông hoặc flagship, bỏ qua phân khúc tầm trung ít được công bố giá.

- **Đề xuất:** Thu thập thêm dữ liệu giá từ các nguồn khác (Amazon, Newegg, Ebay) để bổ sung các quan sát thiếu giá, hoặc áp dụng phương pháp **Multiple Imputation** cho biến phụ thuộc.

Hạn chế 4 — Không xét tương tác giữa biến: Mô hình hiện tại giả định các biến tác động độc lập lên giá. Trên thực tế, tác động của TDP lên giá có thể phụ thuộc vào hãng sản xuất (ví dụ: cùng TDP nhưng NVIDIA định giá khác AMD).

- **Đề xuất:** Thêm các **số hạng tương tác** (interaction terms) như `tdp:manufacturer` vào mô hình và sử dụng Ramsey RESET test để kiểm tra sai dạng hàm.

5.3 Ý nghĩa thực tiễn

Mô hình Log-Linear đưa ra những ứng dụng thực tiễn quan trọng:

(1) **Công cụ định giá cho người mua:** Với phương trình $\log(\hat{Y}) = 84.45 + 0.455 \cdot \text{TDP} + \dots$, người tiêu dùng có thể ước lượng giá hợp lý của một GPU dựa trên thông số kỹ thuật của nó. Nếu giá thực tế cao hơn đáng kể so với giá mô hình dự đoán, đó là dấu hiệu GPU đang bị định giá cao (overpriced) và người mua nên cân nhắc thay thế.

(2) **Định lượng “phí thương hiệu” NVIDIA:** Kết quả cho thấy với cùng thông số kỹ thuật, GPU NVIDIA đắt hơn AMD khoảng **45.2%**. Đây là con số cụ thể giúp người mua GPU đánh giá liệu mức giá premium của NVIDIA có xứng đáng với nhu cầu của mình không, đặc biệt trong bối cảnh cả hai hãng đều có GPU trên cùng một phân khúc hiệu năng.

(3) **Hỗ trợ hoạch định ngân sách doanh nghiệp:** Đối với các doanh nghiệp mua số lượng lớn GPU cho các hệ thống AI hoặc render farm, mô hình cung cấp cơ sở khoa học để so sánh giá trị thực tế của từng sản phẩm trong danh mục, tránh bị tác động bởi marketing và tập trung vào hiệu năng-trên-dô-la.

(4) **Tín hiệu cho nhà sản xuất:** Hệ số âm của `release_year` ($\beta = -0.039$) cho thấy GPU ra mắt sau rẻ hơn $\approx 3.8\%$ mỗi năm — phản ánh xu hướng giảm giá theo thời gian của công nghệ phần cứng (Moore’s Law). Đây là thông tin hữu ích cho chiến lược định giá và thời điểm ra mắt sản phẩm.

Tóm lại, nghiên cứu đã thành công trong việc xây dựng một mô hình thống kê có cơ sở khoa học vững chắc và có giá trị ứng dụng thực tiễn cho cả người tiêu dùng lẫn nhà sản xuất GPU trên thị trường.

Tài liệu

- [1] Ilias Kkaf. (2017). *Computer Parts (CPUs and GPUs)*. Kaggle. <https://www.kaggle.com/datasets/iliassekkaf/computerparts>. Truy cập lần cuối: tháng 4/2026.
- [2] Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to Linear Regression Analysis* (5th ed.). John Wiley & Sons.
- [3] R Core Team. (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>.



- [4] Kutner, M. H., Nachtsheim, C. J., Neter, J., & Li, W. (2004). *Applied Linear Statistical Models* (5th ed.). McGraw-Hill/Irwin.
- [5] Field, A. (2018). *Discovering Statistics Using IBM SPSS Statistics* (5th ed.). SAGE Publications.